
Теория: скалярное автоматическое дифференцирование и обучение одного нейрона

Содержание

Как читать обозначения	1
1. Функция, приращение и производная	1
2. Частные производные и градиент	2
3. Правило цепочки: связь между соседними шагами	2
4. Вычислительный граф и прямой проход	3
5. Локальные производные: почему нужны правила для каждой операции	3
Сложение	7
Умножение	8
Отрицание и вычитание	8
Степень с постоянным показателем	9
Синус, экспонента, логарифм и гиперболический тангенс	9
6. От локальных правил к полному обратному проходу	10
Цепочка: $J = \sin(x^2)$	10
Ветвящийся граф: $J = \tanh(ab + a)$	11
Что меняется по сравнению с одной цепочкой	11
Сверка обратного прохода с обычным дифференцированием	15
Граф данных и схема обратного прохода — это не два разных графа	16
7. Топологический порядок, множество visited и начальный градиент	16
8. Центральная разность как независимая проверка	18
9. Один нейрон, функция потерь и общий градиент параметров	18
10. Ограничения учебной реализации	20

Эта теория сопровождает лабораторную работу «Скалярное автоматическое дифференцирование и обучение одного нейрона». Она начинается с обычной производной и последовательно приводит к коду, который автоматически получает градиенты параметров. Здесь нет библиотек машинного обучения, векторов или матриц: каждая вершина графа хранит одно вещественное число. Благодаря этому можно увидеть все связи между формулами и строками программы.

Как читать обозначения

В первой, учебной части итоговую скалярную функцию обозначаем буквой J . Это может быть любая величина, градиент которой нам нужен. В части про обучение нейрона буква L обозначает именно функцию потерь. Такое разделение не позволяет смешать произвольный итог вычислительного графа и ошибку модели.

Для любой вершины v определим краткое обозначение:

$$g_v = \frac{\partial J}{\partial v}.$$

Число g_v — полный накопленный градиент итоговой функции J по вершине v . В программе ему соответствует поле `grad`. Если текущая операция z добавляет только часть полного градиента родителя x , мы обозначим этот один вклад так:

$$\Delta g_x^{(z)}.$$

Индекс (z) означает: «вклад, пришедший в x именно через операцию, результатом которой является z ». После сложения всех таких вкладов получается полный g_x .

1. Функция, приращение и производная

Функция получает вход и возвращает результат. Для скалярной функции вход и результат являются одним числом:

$$y = f(x).$$

Производная отвечает на вопрос: насколько изменится y , если изменить x очень немного. Формальное определение использует предел:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Здесь h — маленькое приращение входа. Дробь сравнивает значение функции после изменения и до изменения. Предел означает, что мы мысленно уменьшаем h до нуля и изучаем, к какому числу стремится это отношение.

Для функции

$$f(x) = x^2$$

производная равна

$$f'(x) = 2x.$$

В точке $x = 3$ производная равна 6. Поэтому при небольшом изменении $\Delta x = 0.01$ результат увеличится приблизительно на $6 \cdot 0.01 = 0.06$. Это выражает локальная линейная аппроксимация:

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x.$$

Слово «локальная» означает, что утверждение относится к небольшой окрестности выбранной точки. Чем меньше Δx , тем точнее приближение для гладкой функции.

2. Частные производные и градиент

Нейрон зависит не от одного числа, а сразу от нескольких. Пусть x — вход, w — вес, b — смещение:

$$z = wx + b,$$

$$\hat{y} = \tanh(z).$$

Если мы хотим узнать влияние только веса w , то временно считаем x и b постоянными. Такая производная называется частной:

$$\frac{\partial \hat{y}}{\partial w}.$$

Символ ∂ отличает частную производную от производной функции одной переменной. Аналогично существуют:

$$\frac{\partial \hat{y}}{\partial b} \quad \text{и} \quad \frac{\partial \hat{y}}{\partial x}.$$

Когда итоговая скалярная функция зависит от многих параметров, набор её частных производных называется градиентом. В обучении нам особенно важны:

$$\frac{\partial L}{\partial w} \quad \text{и} \quad \frac{\partial L}{\partial b}.$$

Они говорят, как изменится функция потерь при малом изменении веса или смещения.

3. Правило цепочки: связь между соседними шагами

Сложная формула обычно строится из нескольких простых операций. Пусть сначала вычисляется промежуточное значение u , а потом итог:

$$u = g(x),$$

$$J = h(u).$$

Тогда влияние x на J проходит через u . Правило цепочки гласит:

$$\frac{dJ}{dx} = \frac{dJ}{du} \frac{du}{dx}.$$

Первый множитель отвечает на вопрос «как итог меняется при изменении промежуточной вершины». Второй множитель отвечает на вопрос «как промежуточная вершина меняется при изменении исходного входа». Их произведение связывает итог с исходным входом.

Для трёх шагов правило просто продолжается:

$$v = f(x),$$

$$u = g(v),$$

$$J = h(u),$$

$$\frac{dJ}{dx} = \frac{dJ}{du} \frac{du}{dv} \frac{dv}{dx}.$$

Автоматическое дифференцирование не выводит такую длинную формулу заново вручную для каждой модели. Оно сохраняет короткое правило каждой операции, а затем систематически применяет правило цепочки.

4. Вычислительный граф и прямой проход

Рассмотрим выражение:

$$m = ab,$$

$$s = m + a,$$

$$J = \tanh(s).$$

Возьмём числа $a = 0.5$ и $b = 1$. Прямой проход даёт:

$$m = 0.5 \cdot 1 = 0.5,$$

$$s = 0.5 + 0.5 = 1,$$

$$J = \tanh(1) \approx 0.761594.$$

Вычислительный граф записывает это вычисление как вершины и направленные рёбра. Вершина хранит число. Ребро означает, что родительское значение использовалось для создания дочернего значения.

Каждый объект Value в лабораторной хранит:

- data — значение вершины, полученное в прямом проходе;
- grad — накопленный градиент итоговой функции по этой вершине;
- parents — непосредственные родители вершины;
- op — имя операции, полезное при чтении графа;
- _backward — локальное правило, которое позднее передаст градиент родителям.

Граф ориентирован: значения данных движутся от родителей к потомкам. Он ациклический: путь не может вернуться в уже созданную вершину. Одна вершина может быть использована более одного раза. В примере a одновременно является входом умножения и вторым слагаемым сложения.

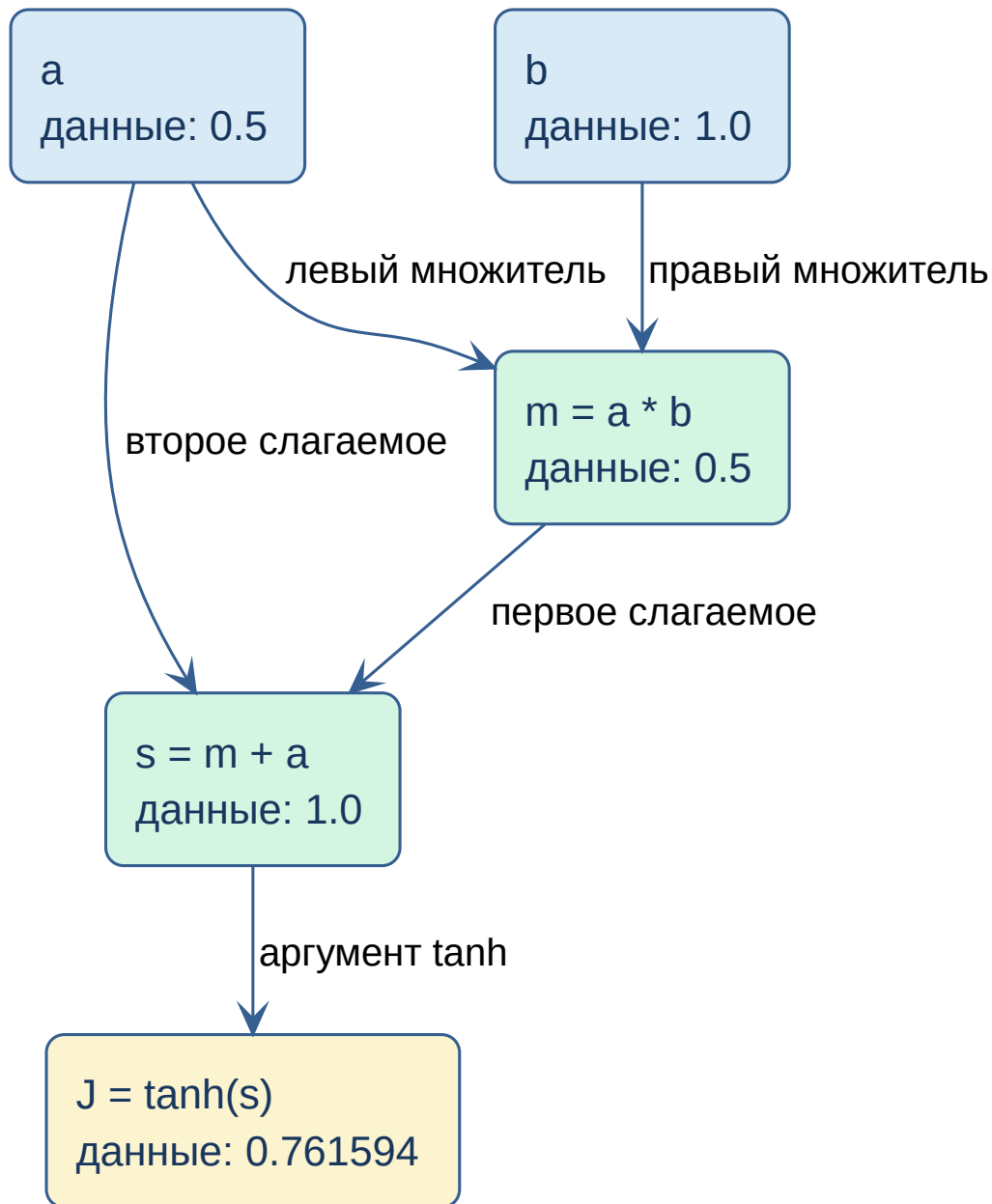
5. Локальные производные: почему нужны правила для каждой операции

До этого момента мы говорили о производной функции целиком. Но код не видит формулу целиком как одну строку. Он видит последовательность маленьких операций. Поэтому ему нужно знать, что делать на одном узле графа, а затем уметь соединить результаты этих маленьких правил.

Пусть один узел графа вычисляет:

$$z = f(x, y),$$

Прямой проход: $J = \tanh(a * b + a)$



Цвет	Смысл
голубой	исходное скалярное значение
зелёный	промежуточный результат
жёлтый	итоговая скалярная функция J

Рис. 1. Прямой вычислительный граф

а всё, что находится после него в графе, можно временно представить одной внешней функцией:

$$J = H(z).$$

Нас интересует влияние x на окончательный результат J . Правило цепочки даёт:

$$\frac{\partial J}{\partial x} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial x}.$$

Аналогично:

$$\frac{\partial J}{\partial y} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial y}.$$

Первый множитель уже несёт информацию обо всём графе после z :

$$g_z = \frac{\partial J}{\partial z}.$$

Второй множитель зависит только от текущей операции f и её непосредственных входов. Именно поэтому производная $\frac{\partial z}{\partial x}$ называется локальной: чтобы найти её, не надо знать весь граф до J .

Один локальный узел внутри большой функции

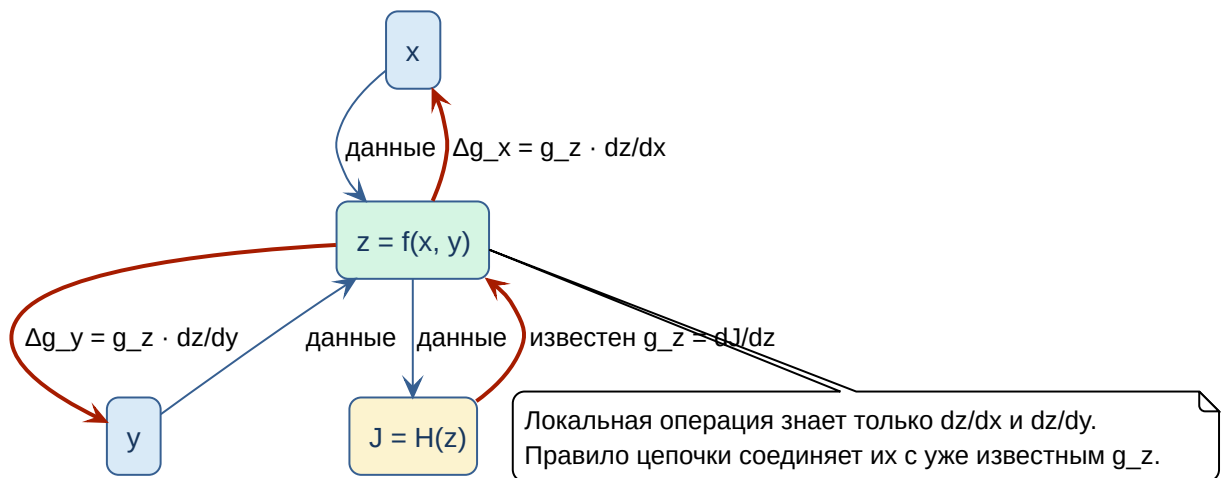


Рис. 2. Общее локальное правило цепочки

В момент обработки узла z в обратном проходе значение g_z уже известно. Это не дополнительное предположение и не число, которое программа угадывает. Обратный проход начинается у итоговой вершины с $g_J = 1$ и движется против направления рёбер данных. Каждый потомок z сначала применяет своё локальное правило и прибавляет в $z.\text{grad}$ собственный вклад. Лишь после обработки всех потомков наступает очередь самого z . Поэтому к этому моменту $z.\text{grad}$ уже содержит сумму всех путей от z к J , то есть полный $g_z = \frac{\partial J}{\partial z}$. Раздел 7 покажет, как обратный топологический порядок гарантирует именно такую последовательность для любого графа.

На схеме p и q — два потомка узла z . В реальном графе потомков может быть один, два или больше.

1. Итоговая функция J задаёт начальные вклады своим непосредственным родителям p и q .
2. Каждый потомок обрабатывается раньше z и прибавляет свой вклад в один и тот же $z.\text{grad}$.

Почему g_z известен до обработки узла z

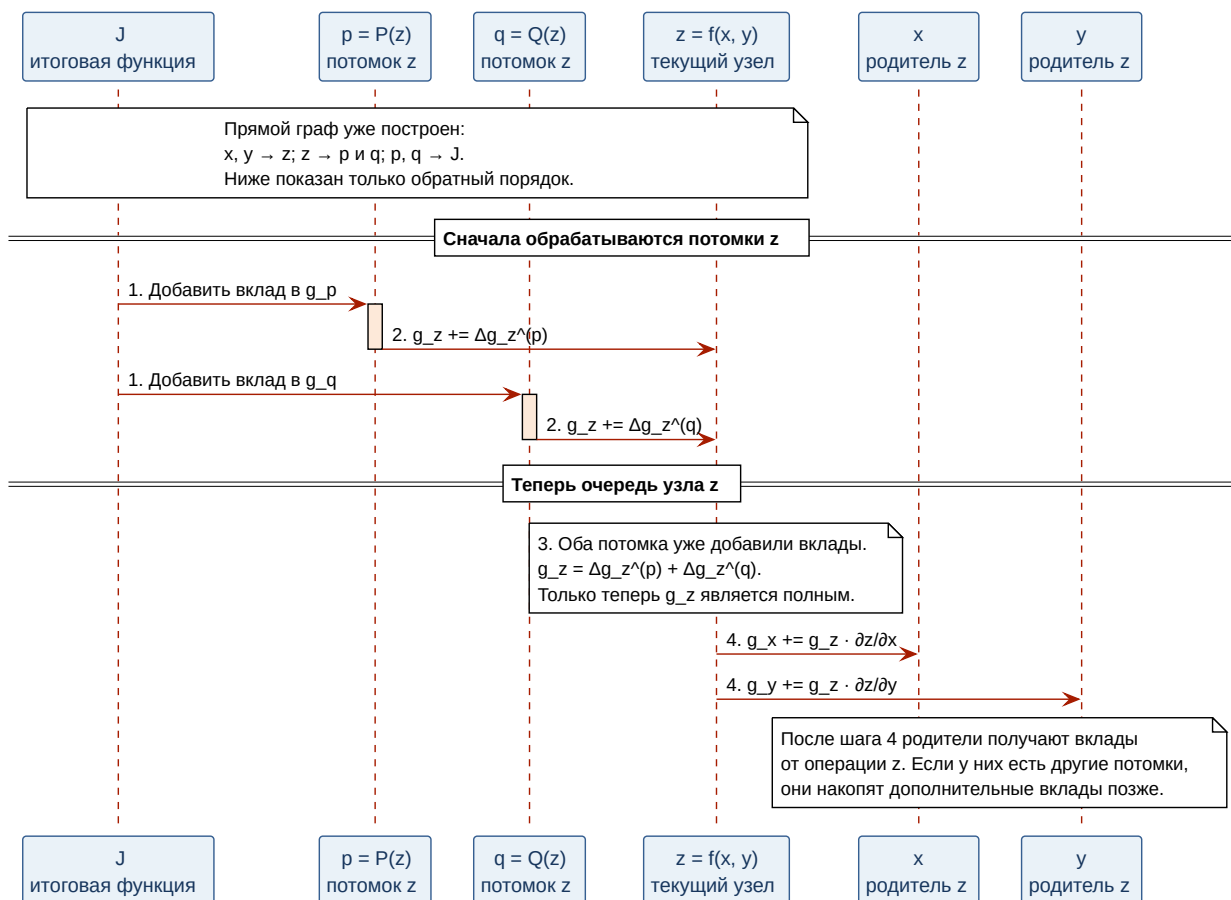


Рис. 3. Почему входящий градиент узла уже собран

3. Когда очередь дошла до z , поле $z.grad$ уже равно $\Delta g_z^{(p)} + \Delta g_z^{(q)}$, то есть полному g_z .
4. Только теперь z применяет собственное локальное правило и передаёт уже полный градиент родителям x и y .

Текущая операция $z = f(x, y)$ вычисляет не новый полный градиент родителя, а один вклад:

$$\Delta g_x^{(z)} = g_z \frac{\partial z}{\partial x},$$

$$\Delta g_y^{(z)} = g_z \frac{\partial z}{\partial y}.$$

Если бы x использовался только в этой одной операции, можно было бы написать равенство $g_x = \Delta g_x^{(z)}$. Но x может участвовать в нескольких операциях. Поэтому правильное общее действие имеет вид:

$$g_x \leftarrow g_x + \Delta g_x^{(z)},$$

$$g_y \leftarrow g_y + \Delta g_y^{(z)}.$$

В коде это реализуется оператором `+=`. Он не является новой производной и не меняет правило цепочки. Он просто складывает вклады разных путей в один полный градиент вершины.

Когда мы создаём объект `result` в прямом проходе, число `result.grad` ещё равно нулю. Вложенная функция `backward` сохраняется заранее вместе с локальной формулой. Позднее, во время метода `backward` всего графа, потомки запишут в `result.grad` уже найденный g_{result} . Только после этого сохранённая функция `backward` выполнит умножение на локальную производную и прибавит вклад родителю.

Сложение

Для операции

$$z = x + y$$

локальные производные равны:

$$\frac{\partial z}{\partial x} = 1,$$

$$\frac{\partial z}{\partial y} = 1.$$

Подставим их в общую формулу. Если в узле z накоплен входящий градиент g_z , то вклад операции сложения равен:

$$\Delta g_x^{(z)} = g_z \cdot 1,$$

$$\Delta g_y^{(z)} = g_z \cdot 1.$$

Смысл прост: изменение каждого слагаемого на маленькую величину меняет сумму на такую же маленькую величину. Если, например, $g_z = 4$, то изменение x на 0.01 меняет J приблизительно на $4 \cdot 0.01$. Поэтому операция добавляет 4 в градиент x и 4 в градиент y .

В коде локальная функция использует:

```
self.grad += result.grad
other.grad += result.grad
```

Здесь `result.grad` хранит g_z , а множитель 1 опущен, потому что умножение на единицу ничего не меняет.

Умножение

Для операции

$$z = xy$$

локальные производные равны:

$$\frac{\partial z}{\partial x} = y,$$

$$\frac{\partial z}{\partial y} = x.$$

Поэтому при известном g_z операция добавляет:

$$\Delta g_x^{(z)} = g_z y,$$

$$\Delta g_y^{(z)} = g_z x.$$

Почему появляется второй множитель? Если y временно фиксирован, выражение xy — это число y , умноженное на x . Например, при $y = -3$ увеличение x на 0.01 уменьшает z на 0.03. Поэтому локальная чувствительность по x равна -3 .

Пусть $x = 2$, $y = -3$, а $g_z = 4$. Тогда:

$$\Delta g_x^{(z)} = 4 \cdot (-3) = -12,$$

$$\Delta g_y^{(z)} = 4 \cdot 2 = 8.$$

В коде сохранённые значения обоих множителей доступны как `self.data` и `other.data`:

```
self.grad += result.grad * other.data
```

```
other.grad += result.grad * self.data
```

Отрицание и вычитание

Для отрицания

$$z = -x$$

локальная производная равна:

$$\frac{dz}{dx} = -1.$$

Поэтому вклад равен:

$$\Delta g_x^{(z)} = g_z(-1).$$

Вычитание не требует отдельного правила, потому что:

$$x - y = x + (-y).$$

Сначала граф создаёт вершину отрицания для y , затем вершину сложения. Обратный проход применяет два уже известных локальных правила.

Степень с постоянным показателем

Для

$$z = x^p,$$

где p — обычная постоянная, локальная производная равна:

$$\frac{dz}{dx} = px^{p-1}.$$

Поэтому вклад операции в родительский градиент:

$$\Delta g_x^{(z)} = g_z px^{p-1}.$$

Показатель p в учебной реализации не является объектом Value. Мы не строим граф и градиент по p .

Синус, экспонента, логарифм и гиперболический тангенс

Все эти операции имеют один родитель x . Если их результат обозначить z , то общий вид обратного правила один:

$$\Delta g_x^{(z)} = g_z \frac{dz}{dx}.$$

Для синуса:

$$z = \sin(x),$$

$$\Delta g_x^{(z)} = g_z \cos(x).$$

Для экспоненты:

$$z = \exp(x),$$

$$\Delta g_x^{(z)} = g_z \exp(x) = g_z z.$$

Сохранённый результат z уже есть в прямом проходе, поэтому код использует `result.data`. Для натурального логарифма при $x > 0$:

$$z = \log(x),$$

$$\Delta g_x^{(z)} = g_z \frac{1}{x}.$$

Для гиперболического тангенса:

$$z = \tanh(x),$$

$$\Delta g_x^{(z)} = g_z (1 - \tanh^2(x)) = g_z (1 - z^2).$$

Гиперболический тангенс возвращает число в интервале от -1 до 1 . Поэтому он удобен для учебной классификации с целями -1 и 1 .

6. От локальных правил к полному обратному проходу

Локальное правило объясняет один шаг. Полный обратный проход применяет такие шаги ко всем вершинам в нужном порядке. Сначала разберём два разных вида графов: цепочку без ветвления и граф с общей вершиной.

Цепочка: $J = \sin(x^2)$

Пусть:

$$u = x^2,$$

$$J = \sin(u),$$

$$x = 0.5.$$

Прямой проход даёт:

$$u = 0.25,$$

$$J = \sin(0.25) \approx 0.247404.$$

Обратный проход начинается с:

$$g_J = 1.$$

Синус передаёт градиент к u :

$$g_u = g_J \cos(u) = \cos(0.25) \approx 0.968912.$$

Это число уже имеет точный смысл: $g_u = \frac{\partial J}{\partial u}$. Если мысленно увеличить u на очень малую величину, например на 0.01, то при неизменных остальных условиях J увеличится приблизительно на $0.968912 \cdot 0.01$. Теперь надо перенести это влияние через предыдущую операцию $u = x^2$.

Для конкретного значения $x = 0.5$ её локальная производная равна:

$$\frac{\partial u}{\partial x} = 2x = 2 \cdot 0.5 = 1.$$

Это означает, что малое изменение x в этой точке вызывает такое же по величине малое изменение u : приращение x сначала умножается на 1, а затем влияние u на J уже измеряет g_u . Поэтому по правилу цепочки получаем:

$$g_x = \frac{\partial J}{\partial x} = g_u \frac{\partial u}{\partial x} = 0.968912 \cdot 1 = 0.968912.$$

Итак, 0.968912 — итоговый градиент по x для этого выражения. После обратного прохода именно это число должно находиться в `x.grad`. Его можно проверить и прямым дифференцированием всей функции:

$$\frac{d}{dx} \sin(x^2) = \cos(x^2) \cdot 2x,$$

$$\left. \frac{d}{dx} \sin(x^2) \right|_{x=0.5} = \cos(0.25) \cdot 1 \approx 0.968912.$$

В этой цепочке у x нет других потомков: к нему приходит только один вклад от узла u . В коде этот шаг выглядит так:

```
x.grad += u.grad * (2 * x.data)
```

Перед этой строкой $x.grad$ равен 0.0 , а $u.grad$ равен 0.968912 . Поэтому для данного примера выполняется обычное численное сложение:

```
x.grad = 0.0 + 0.968912 * 1
```

и в $x.grad$ получается 0.968912 . Здесь $+=$ действительно дал бы то же число, что и присваивание $x.grad = u.grad * (2 * x.data)$, но только потому, что прежнее значение равно нулю и вклад единственный. Мы всё равно используем $+=$ как общее правило: в следующем примере у одной вершины будет два потомка, и тогда два разных вклада необходимо сложить, а не заменить один другим.

Ветвящийся граф: $J = \tanh(ab + a)$

Теперь вернёмся к графу из раздела 4:

$$m = ab,$$

$$s = m + a,$$

$$J = \tanh(s).$$

Что меняется по сравнению с одной цепочкой

В предыдущем примере $J = \sin(x^2)$ была единственная цепочка влияния $x \rightarrow u \rightarrow J$. Поэтому производная по x получалась одним произведением локальных производных. В новом графе операции всё ещё скалярные, но некоторые из них имеют два входа, а одна и та же вершина a используется в двух местах выражения. Это и делает граф ветвящимся:

$$a \rightarrow s \rightarrow J,$$

$$a \rightarrow m \rightarrow s \rightarrow J,$$

$$b \rightarrow m \rightarrow s \rightarrow J.$$

Стрелки здесь обозначают направление вычисления данных. У переменной a есть два разных пути к итоговой функции: прямой путь через слагаемое $+a$ и путь через произведение $m = ab$. У b такой путь один. Следовательно, итоговая производная по a должна учитывать влияние по обоим путям, а итоговая производная по b — по одному.

Напомним соглашение о записи. Для любой вершины v вычислительного графа $g_v = \frac{\partial J}{\partial v}$ означает влияние изменения значения именно этой вершины на итог J при фиксированных остальных входах её последующего подграфа. Поэтому g_s и g_m — градиенты промежуточных вершин. Листья a и b являются независимыми входами исходной функции $J(a, b)$. После суммирования всех путей их значения равны частным производным при фиксированном другом входе:

$$g_a = \frac{\partial J}{\partial a},$$

$$g_b = \frac{\partial J}{\partial b}.$$

Это частный случай многомерного правила цепочки. Если независимый вход t влияет на J через несколько промежуточных величин $u_1, \dots, u_k$, то частная производная по t складывает влияния всех этих маршрутов:

$$\frac{\partial J}{\partial t} = \sum_{i=1}^k \frac{\partial J}{\partial u_i} \frac{\partial u_i}{\partial t}.$$

В нашем выражении удобно начать от s . Переменная a влияет на s двумя способами: непосредственно как второе слагаемое и косвенно через $m = ab$. Поэтому обычное правило цепочки с суммой даёт:

$$\frac{\partial J}{\partial a} = \frac{\partial J}{\partial s} \left(\frac{\partial s}{\partial a} + \frac{\partial s}{\partial m} \frac{\partial m}{\partial a} \right).$$

Подставляя локальные производные $\frac{\partial s}{\partial a} = 1$, $\frac{\partial s}{\partial m} = 1$ и $\frac{\partial m}{\partial a} = b$, получаем:

$$\frac{\partial J}{\partial a} = g_s(1 + 1 \cdot b) = g_s(1 + b).$$

Для b существует только маршрут через m , поэтому слагаемое одно:

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial s} \frac{\partial s}{\partial m} \frac{\partial m}{\partial b} = g_s \cdot 1 \cdot a = g_s a.$$

Автоматическое дифференцирование не будет заранее раскрывать выражение $ab + a$ и не будет вручную перечислять пути. Прямой проход сохранит три узла m , s и J , а у каждого узла — его непосредственных родителей и локальное правило. Обратный проход выполнит эти локальные правила в обратном топологическом порядке. Два независимых локальных вклада попадут в одно поле `a.grad` через `+=`; их сумма и будет значением $\frac{\partial J}{\partial a}$. Это программная реализация суммы в многомерном правиле цепочки.

При $a = 0.5$ и $b = 1$ прямой проход уже дал:

$$m = 0.5,$$

$$s = 1,$$

$$J \approx 0.761594.$$

Ниже показана именно последовательность обратного прохода для этого числового примера. Это не второй вычислительный граф: исходные зависимости уже сохранены после прямого прохода. Каждая стрелка на схеме означает один локальный вклад Δg в градиент родительской вершины. Порядок стрелок важен: сначала узел s получает градиент от J , затем передаёт его своим родителям; только после этого узел m передаёт второй вклад в общую вершину a .

Обратите внимание на два разных момента, которые диаграмма намеренно разделяет. После стрелки $s \rightarrow a$ в `a.grad` уже записано число 0.419974, однако это ещё не окончательный g_a : второй потомок m ещё не передал свой вклад. После стрелки $m \rightarrow a$ оператор `+=` складывает оба вклада, и только тогда получается полный градиент $g_a \approx 0.839949$. Далее тот же ход выводится формула за формулой.

Начинаем с $g_J = 1$. Гиперболический тангенс передаёт:

$$g_s = g_J(1 - J^2) = 1 \cdot (1 - 0.761594^2) \approx 0.419974.$$

Разберём теперь узел сложения $s = m + a$. Почему от него выходят две стрелки обратного прохода? В прямом проходе этот узел получил два непосредственных входа: уже вычисленное значение m и значение a . Поэтому у операции есть две локальные частные производные.

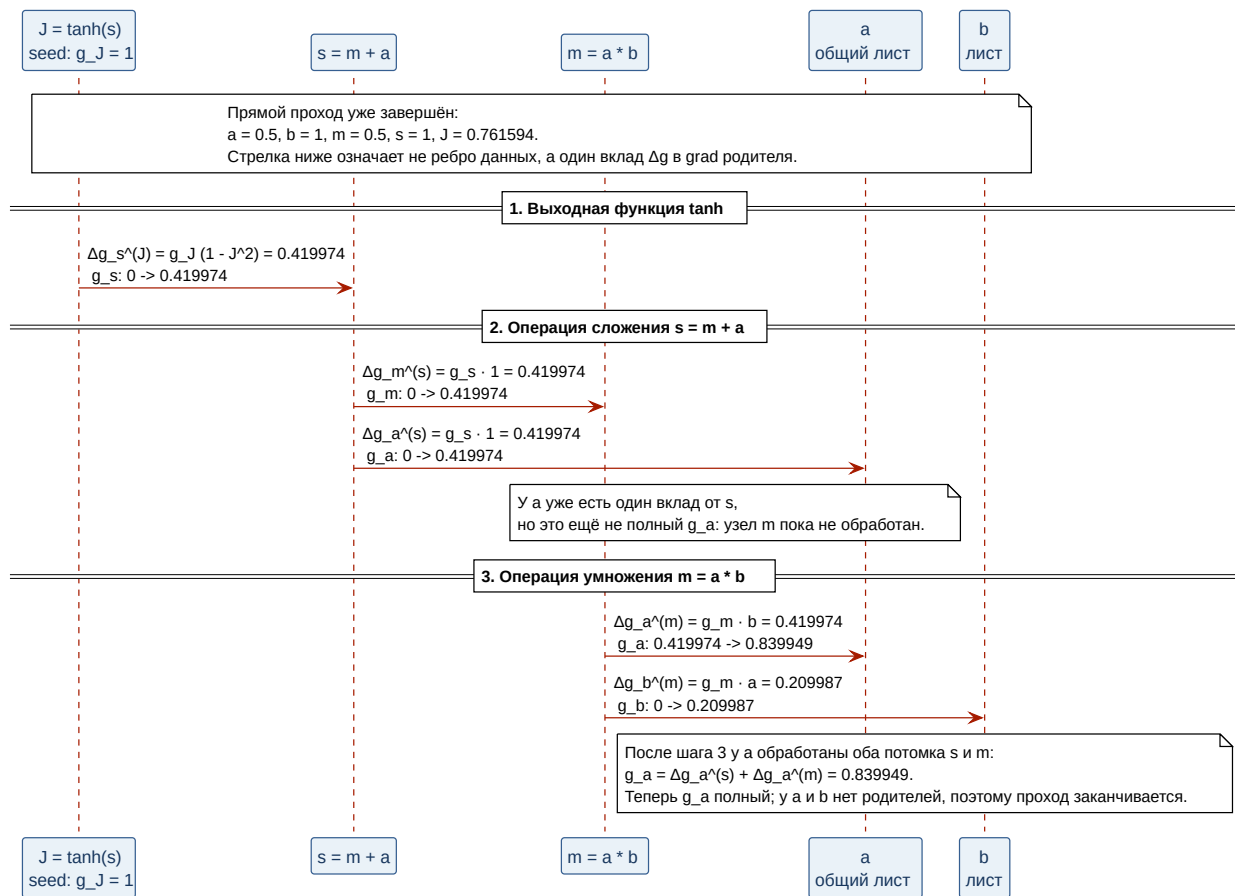
Строгая последовательность обратного прохода: $J = \tanh(ab + a)$ 

Рис. 4. Последовательность обратного прохода ветвящегося графа

Когда вычисляем первую, считаем a фиксированным и меняем только m ; когда вычисляем вторую, считаем m фиксированным и меняем только a :

$$\frac{\partial s}{\partial m} = 1,$$

$$\frac{\partial s}{\partial a} = 1.$$

Важно не подменять в этой локальной операции m выражением ab . Для узла сложения m и a являются его непосредственными входами, поэтому при нахождении $\frac{\partial s}{\partial a}$ значение m временно считается независимым и фиксированным. Зависимость m от a будет учтена отдельно, когда обратный проход дойдёт до узла умножения $m = ab$. Так один путь $a \rightarrow s \rightarrow J$ и другой путь $a \rightarrow m \rightarrow s \rightarrow J$ учитываются по одному разу.

Значение $g_s = \frac{\partial J}{\partial s} \approx 0.419974$ уже говорит, как изменение s влияет на итог J . Умножаем его на каждую локальную производную сложения. Получаются не два вклада в s , а по одному новому вкладу для каждого из родителей узла s :

$$\Delta g_m^{(s)} = g_s \frac{\partial s}{\partial m} = 0.419974 \cdot 1 = 0.419974,$$

$$\Delta g_a^{(s)} = g_s \frac{\partial s}{\partial a} = 0.419974 \cdot 1 = 0.419974.$$

До этого шага поля `m.grad` и `a.grad` равны нулю. Поэтому после обработки сложения происходят именно такие обновления:

```
m.grad += 0.419974 # было 0.0, стало примерно 0.419974
a.grad += 0.419974 # было 0.0, стало примерно 0.419974
```

У m в этом графе только один потомок, s , поэтому его градиент уже полный:

$$g_m = \frac{\partial J}{\partial m} = 0.419974.$$

У a ситуация другая. Число 0.419974 в `a.grad` пока отражает только прямой путь $a \rightarrow s \rightarrow J$. Узел a ещё участвует в умножении $m = ab$, а значит после обработки m в `a.grad` появится второй вклад.

Теперь обратный проход обрабатывает умножение $m = ab$. Его непосредственные родители — a и b . Если удерживать b фиксированным и увеличить a на малую величину, то m увеличится примерно на b раз эту величину; поэтому:

$$\frac{\partial m}{\partial a} = b = 1.$$

Если удерживать a фиксированным и менять b , то коэффициентом будет a :

$$\frac{\partial m}{\partial b} = a = 0.5.$$

В оба правила подставляем уже найденный полный градиент $g_m = 0.419974$:

$$\Delta g_a^{(m)} = g_m \frac{\partial m}{\partial a} = 0.419974 \cdot 1 = 0.419974,$$

$$\Delta g_b^{(m)} = g_m \frac{\partial m}{\partial b} = 0.419974 \cdot 0.5 = 0.209987.$$

Теперь можно увидеть работу накопления буквально. В `a.grad` уже лежит вклад от сложения, поэтому второй вклад не заменяет первое число:

```
a.grad += 0.419974 # было примерно 0.419974, стало примерно 0.839949
b.grad += 0.209987 # было 0.0, стало примерно 0.209987
```

Итоговые градиенты листьев равны:

$$g_a = \Delta g_a^{(s)} + \Delta g_a^{(m)} \approx 0.419974 + 0.419974 \approx 0.839949,$$

$$g_b = \Delta g_b^{(m)} \approx 0.209987.$$

Сверка обратного прохода с обычным дифференцированием

Теперь проверим, что пошаговый обратный проход получил не просто похожие числа, а в точности те же производные, которые даёт обычное дифференцирование исходной функции двух независимых переменных:

$$J(a, b) = \tanh(ab + a).$$

Сначала дифференцируем внутреннее выражение по a . Здесь b фиксировано, поэтому:

$$\frac{\partial(ab + a)}{\partial a} = b + 1.$$

Производная внешней функции $\tanh(r)$ по её аргументу r равна $1 - \tanh^2(r)$. По обычному правилу цепочки получаем:

$$\frac{\partial J}{\partial a} = (1 - \tanh^2(ab + a)) (b + 1).$$

Для производной по b фиксируем a . У внутреннего выражения остаётся только множитель a :

$$\frac{\partial(ab + a)}{\partial b} = a,$$

$$\frac{\partial J}{\partial b} = (1 - \tanh^2(ab + a)) a.$$

При $a = 0.5$ и $b = 1$ имеем $ab + a = 1$ и $J = \tanh(1)$. Поэтому общий множитель обеих обычных производных равен уже найденному градиенту промежуточной вершины:

$$1 - \tanh^2(ab + a) = 1 - \tanh^2(1) = g_s \approx 0.419974341614.$$

Подстановка даёт:

$$\frac{\partial J}{\partial a} = 0.419974341614 \cdot (1 + 1) \approx 0.839948683228 \approx 0.839949,$$

$$\frac{\partial J}{\partial b} = 0.419974341614 \cdot 0.5 \approx 0.209987170807 \approx 0.209987.$$

Ниже не показан новый порядок вычислений: порядок обратного прохода уже строго задан диаграммой последовательности выше. Эта таблица сопоставляет один и тот же результат двумя способами и завершает проверку.

Величина	Обычное дифференцирование	Соответствующий шаг reverse-mode	Значение при $a = 0.5$, $b = 1$
Общий множитель	$1 - \tanh^2(ab + a)$	$g_s = g_J(1 - J^2)$	0.419974

Величина	Обычное дифференцирование	Соответствующий шаг reverse-mode	Значение при $a = 0.5$, $b = 1$
Прямой путь от a	$(1 - \tanh^2(ab + a)) \cdot \frac{\partial J}{\partial a}$	$\Delta g_a^{(s)} = g_s \frac{\partial s}{\partial a}$	0.419974
Путь от a через m	$(1 - \tanh^2(ab + a)) \cdot \frac{\partial J}{\partial m} \cdot b$	$\Delta g_a^{(m)} = g_m \frac{\partial m}{\partial a}$	0.419974
Итог по a	$\frac{\partial J}{\partial a} = (1 - \tanh^2(ab + a)) (1 + b)$	$g_a = \Delta g_a^{(s)} + \Delta g_a^{(m)}$	0.839949
Итог по b	$\frac{\partial J}{\partial b} = (1 - \tanh^2(ab + a)) a$	$g_b = \Delta g_b^{(m)}$	0.209987

Следовательно, обратный проход не вычисляет другую величину и не является приближением обычного дифференцирования. Он разбивает те же производные на локальные вклады, накапливает их в общих вершинах и получает тот же точный результат с точностью вычислений с плавающей запятой.

Граф данных и схема обратного прохода — это не два разных графа

Прямой и обратный рисунки показывают одни и те же вершины и те же зависимости. В прямом направлении вершина использует data родителей, чтобы получить собственное data. В обратном направлении вершина использует сохранённые data и свой накопленный grad, чтобы добавить вклады в grad родителей.

Обратный рисунок не создаёт вторую модель и не отменяет вычисления прямого прохода. Это способ прочитать тот же ориентированный ациклический граф в другом направлении:

1. прямой проход создаёт вершины, рёбра, значения data и локальные функции `_backward`;
2. обратный проход помещает $g_J = 1$ в итоговую вершину;
3. каждая вершина берёт свой уже накопленный grad;
4. вершина вычисляет локальные вклады и прибавляет их родителям;
5. общая вершина получает сумму вкладов от всех своих потомков.

7. Топологический порядок, множество visited и начальный градиент

Чтобы применить локальное правило вершины, её градиент уже должен быть полностью собран от потомков. Поэтому обратный проход не может просто идти по вершинам в порядке их создания.

Рекурсивный обход `build` сначала посещает родителей, затем добавляет текущую вершину в список `toro`. Для графа $J = \tanh(ab + a)$ получается топологический порядок:

$$[a, b, m, s, J].$$

После разворота списка получаем порядок обратного прохода:

$$[J, s, m, b, a].$$

Ниже приведена строгая диаграмма последовательности. Читать её следует сверху вниз. В первой части показано, как `build` добавляет вершину в `toro` только после всех её родителей. Во второй части показан единственный допустимый порядок вызова локальных функций `_backward`: в момент каждого вызова входящий градиент этой вершины уже накоплен всеми её потомками.

Для нашего графа действия можно записать таблицей.

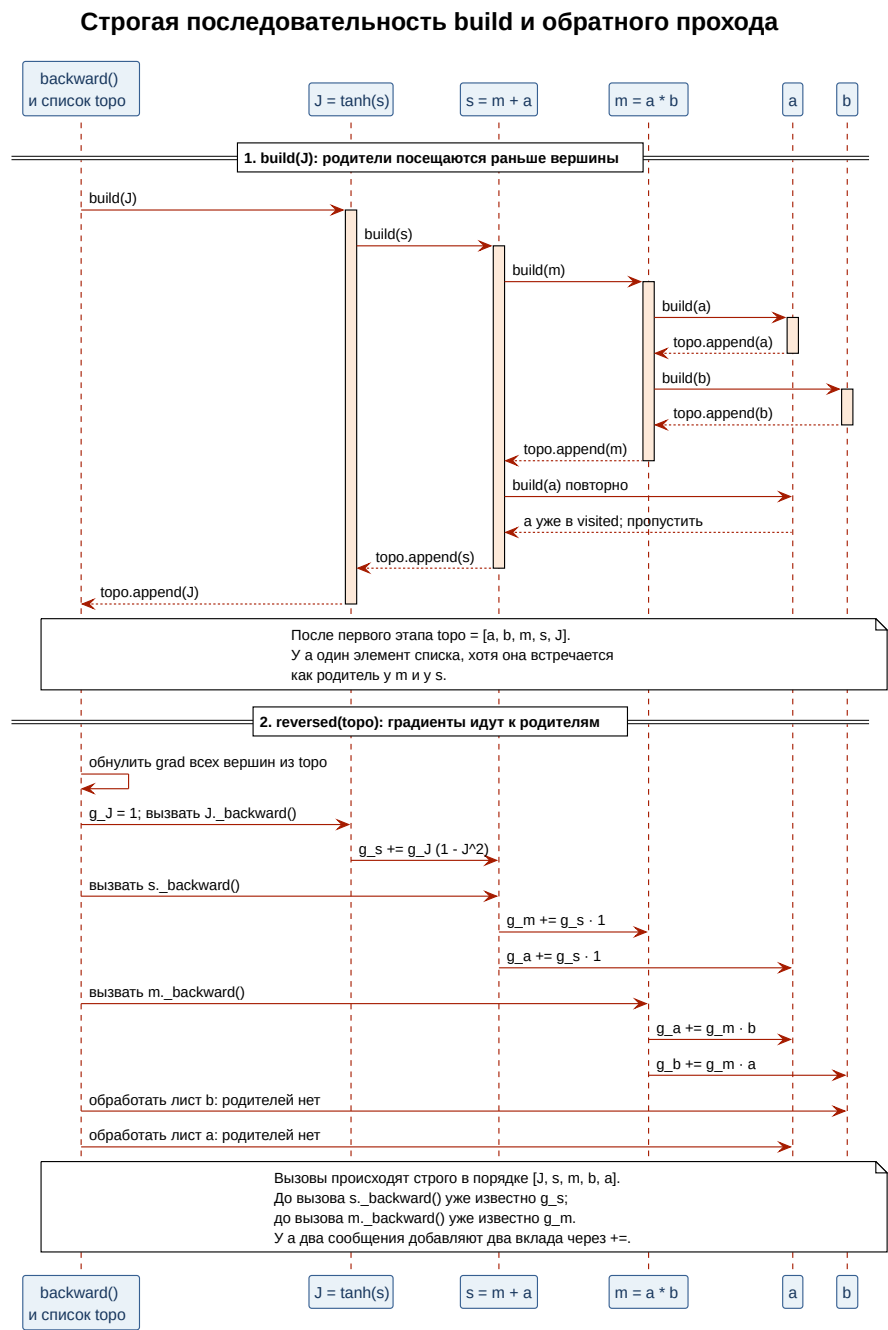


Рис. 5. Последовательность построения topo и обратного прохода

Обрабатываемая вершина	Уже известный grad	Локальное действие	Новые вклады
$J = \tanh(s)$	$g_J = 1$	умножить на $1 - J^2$	добавить g_s в s
$s = m + a$	g_s	умножить на 1 для каждого родителя	добавить g_m в m , добавить первый вклад в a
$m = ab$	g_m	умножить на второй и первый множители	добавить второй вклад в a , добавить вклад в b
b	g_b	листовая вершина	дальнейших родителей нет
a	g_a	листовая вершина	дальнейших родителей нет

Вершина a достижима двумя путями, но должна появиться в топе один раз. Множество visited отмечает, что вершина уже посещена. Оно не отменяет два градиентных вклада: вклады создаются позже локальными правилами и оба прибавляются в одно поле a.grad.

Итоговая вершина получает начальный градиент:

$$g_J = \frac{\partial J}{\partial J} = 1.$$

Это следует из того, что небольшое изменение самой величины J на ΔJ меняет J ровно на ΔJ . До нового запуска backward все grad достижимого подграфа обнуляются. Иначе новый обратный проход прибавил бы новые значения к старому вычислению, которое относится к прежнему запуску.

8. Центральная разность как независимая проверка

Аналитический градиент можно проверить, не доверяя реализации. Центральная разность использует значения функции по обе стороны точки:

$$\frac{df}{dx} \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Мы берём $h = 10^{-6}$ и сравниваем численный результат с результатом Value. Из-за конечной точности вещественных чисел нужно сравнивать не строгое равенство, а абсолютную ошибку:

$$|g_{\text{автоматическое}} - g_{\text{численное}}|.$$

Малая ошибка не заменяет доказательство формул, но хорошо обнаруживает типичные ошибки: неверный множитель при умножении, потерянный знак, пропущенный вклад ветви или неверный порядок обхода.

9. Один нейрон, функция потерь и общий градиент параметров

Один нейрон получает скалярный вход x_i , хранит общий вес w и общее смещение b :

$$z_i = wx_i + b,$$

$$\hat{y}_i = \tanh(z_i).$$

Для каждого объекта целевое значение y_i постоянно. Сначала строится ошибка и её квадрат:

$$e_i = \hat{y}_i - y_i,$$

$$q_i = e_i^2.$$

Для n объектов функция потерь равна:

$$L = \frac{1}{n} \sum_{i=1}^n q_i.$$

Прямой вычислительный граф одного объекта нейрона

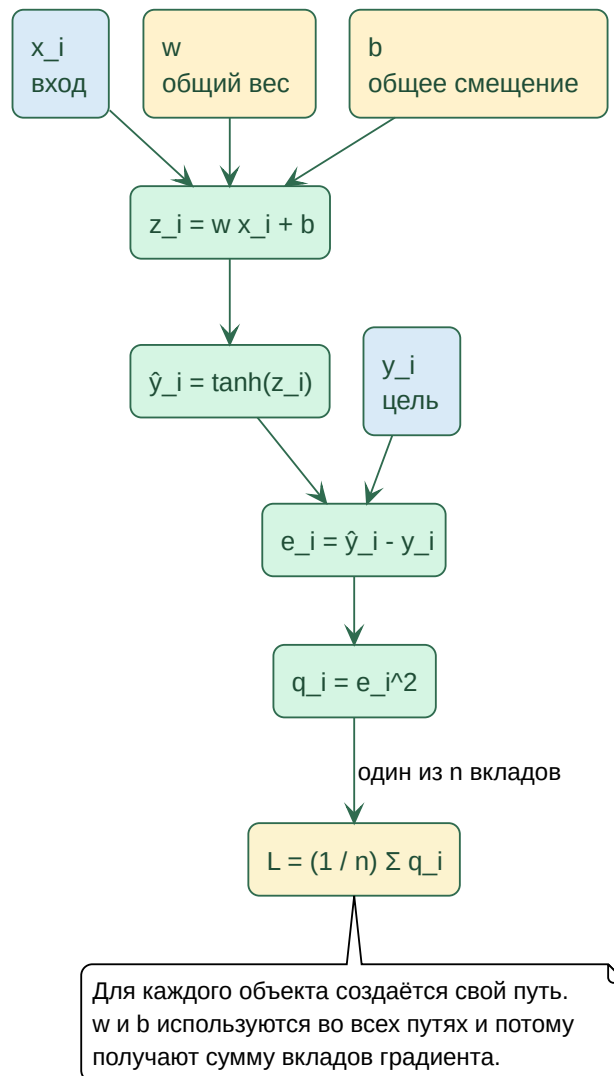


Рис. 6. Прямой граф одного объекта нейрона

Вес и смещение используются при обработке каждого объекта. Поэтому каждый объект создаёт собственный путь к w и b , а поля $w.grad$ и $b.grad$ накапливают суммы вкладов.

Обратный проход начинается с:

$$\frac{\partial L}{\partial L} = 1.$$

Для одного слагаемого q_i среднее передаёт вклад:

$$g_{q_i} += \frac{1}{n}.$$

Квадрат ошибки передаёт:

$$g_{e_i} += g_{q_i} 2e_i.$$

Вычитание передаёт весь градиент прогнозу:

$$g_{\hat{y}_i} += g_{e_i}.$$

Гиперболический тангенс передаёт:

$$g_{z_i} += g_{\hat{y}_i} (1 - \hat{y}_i^2).$$

Линейная комбинация добавляет в общие параметры:

$$g_w += g_{z_i} x_i,$$

$$g_b += g_{z_i}.$$

После обработки всех объектов поля `w.grad` и `b.grad` равны полным производным:

$$g_w = \frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(\hat{y}_i - y_i)(1 - \hat{y}_i^2)x_i,$$

$$g_b = \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(\hat{y}_i - y_i)(1 - \hat{y}_i^2).$$

В лабораторной эти длинные формулы не записываются вручную в цикле обучения. Каждая операция строит маленькую часть графа, а `backward` получает те же производные через локальные правила и правило цепочки.

После вызова `loss.backward` параметры обновляются с положительной скоростью обучения η :

$$w \leftarrow w - \eta \frac{\partial L}{\partial w},$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

Минус нужен потому, что градиент указывает направление наибольшего роста функции, а обучение хочет уменьшить ошибку.

10. Ограничения учебной реализации

Наш `Value` специально остаётся маленьким. Он работает только со скалярами. Он не поддерживает векторы, матрицы, пакетную обработку, `broadcasting`, вторые производные, сохранение графа для повторных производных и профессиональные оптимизаторы. Логарифм определён только при $x > 0$. Показатель степени не является объектом `Value` и не получает градиент.

Эти ограничения не меняют основной принцип. Промышленные библиотеки строят графы для многомерных массивов и эффективно выполняют те же шаги: прямой проход сохраняет

Обратный проход одного объекта к общим параметрам

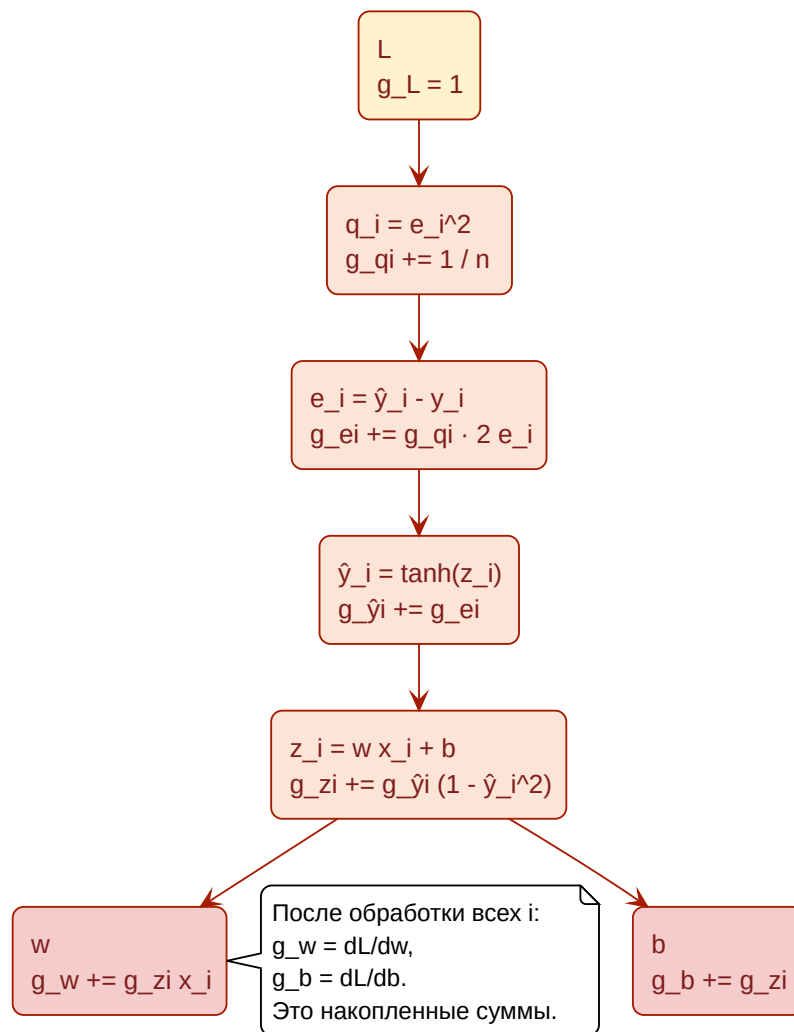


Рис. 7. Обратный граф одного объекта нейрона

Одна эпоха обучения: граф данных, градиентов и обновления

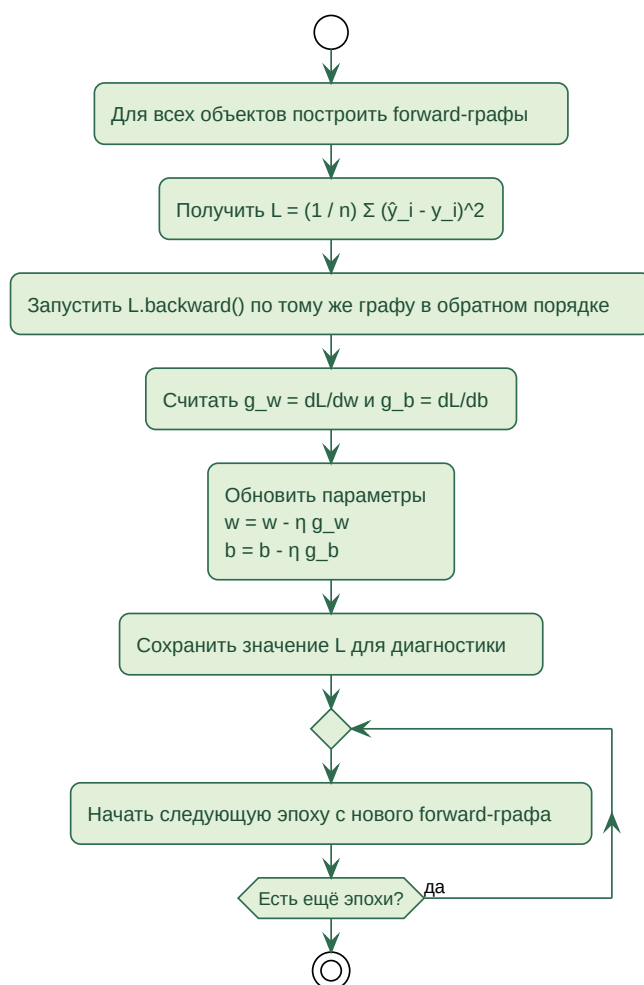


Рис. 8. Цикл обучения нейрона

необходимую информацию, а обратный проход соединяет локальные производные правилом цепочки и суммирует вклады общих параметров.

Теперь переходите к [пошаговой лабораторной работе](#).